Assignment 2: Cost-Efficient Segmentation under a Compute Budget

Jatin Purbia — B23CH1022
CSL7360 — Computer Vision
Indian Institute of Technology Jodhpur

1 November 2026

Abstract

We solve Track B of the assignment with a small U-Net trained from scratch (439,393 parameters, 1.78 MB, no pretrained weights) that labels every pixel of a 256×256 photo as object or background. The network runs internally at 96×96, which keeps the mean single-thread CPU time near 17 ms against the 50 ms budget. Choices were made on a held-out part of DUTS-TR. On the untouched DUTS-TE set the model reaches mIoU 0.705, BF1 0.394, $Q = 0.612$ and CES = 0.544. Metrics on our own phone photographs are pending because their hand-drawn masks have not been supplied.

1 Introduction

The task is binary salient-object segmentation: assign each pixel to the main object(s) or background. The objective is to maximize the Cost-Efficient Segmentation (CES) score subject to a mean single-thread CPU time of at most 50 milliseconds per photo. Region overlap alone is insufficient: the quality term also rewards accurate boundaries, and CES subtracts a small charge for time and file size. We follow Track B (a fully convolutional network). All code uses only the Python standard library, PyTorch, NumPy, OpenCV and Matplotlib, and the metrics are written by hand.

2 Dataset and leakage prevention

DUTS contains 10,553 training photographs (DUTS-TR) and 5,019 test photographs (DUTS-TE) [2]. A sorted list of DUTS-TR stems is shuffled using Python's seed-42 random generator; round($0.1N$) IDs form validation. For the official training size this gives 9,498 training and 1,055 validation IDs. The actual manifests, once prepared, are saved and hashed. Intel(R) Core(TM) 7 150U. Actual saved split: 9498 training and 1055 validation images; seed 42.

All architecture, loss, threshold, resolution and stopping decisions use only DUTS-TR validation. The experiment runner never loads DUTS-TE. Final evaluation requires a frozen parameter-file hash and records all 5,019 samples individually. Personal photos are testing-only and require 5–10 authentic phone captures covering at least three condition categories. A script can validate files and author attestations; it cannot prove photographic authenticity or scene diversity.

3 Preprocessing

OpenCV reads BGR uint8 photographs. Every original photo is directly resized to 256×256 using `INTER_AREA`; aspect ratio is intentionally squashed. Grayscale masks use `INTER_NEAREST` and are binarized at intensity 128 inclusive. The network receives RGB float32 tensors $[B, 3, H, W]$ in $[0, 1]$, with no mean/std normalization. Training augmentation jointly flips image/mask horizontally with probability 0.5 and applies brightness/contrast multipliers sampled uniformly in $[0.9, 1.1]$ to images only. Validation and test preprocessing are deterministic.

The timed pipeline starts after the original resize. It includes BGR-to-RGB conversion, tensor creation, normalization, optional bilinear downscaling, forward inference, bilinear upsampling of logits to 256×256, sigmoid, thresholding and conversion to a binary uint8 array. Masks on disk are multiplied by 255 after timing.

4 Proposed architectures

All candidates are encoder–decoder networks built from convolution, batch normalization, ReLU, max pooling, bilinear upsampling and concatenation skips. Three families were explored in short pilot runs (Tiny FCN: one convolution per stage; U-Net: two per stage; depthwise-separable). The selected model is the U-Net with base width $b = 16$ and four stages: encoder widths $16, 32, 64, 128$, each stage with two Conv–BN–ReLU blocks, 2×2 max pooling between stages, then three decoder stages that bilinearly upsample, concatenate the matching encoder feature map and apply Conv–BN–ReLU. A 1×1 convolution produces one logit per pixel.

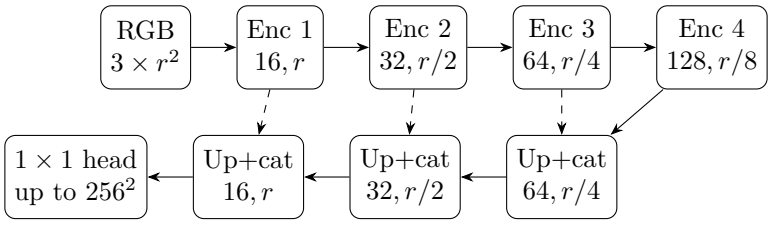


Figure 1: Selected U-Net ($r = 96$ at inference, $r = 128$ during training). Dashed arrows are concatenation skips.

Validation-selected candidate: unet16_bce_128_infer96. Architecture: {'kind': 'unet', 'base': 16, 'depth': 4}; learned parameters: 439393; inference resolution: 96; threshold: 0.5.

5 Mathematical formulation

For input channel c, output channel o and spatial offsets (u, v), convolution is

$$z_{oij} = a_o + \sum_c \sum_u \sum_v W_{ocuv} x_{c,i+u,j+v}. \tag{1}$$

Here W contains learned filters and a is a bias (omitted in normalized blocks). For logit z_i, probability p_i and binary label y_i,

$$p_i = \sigma(z_i) = \frac{1}{1+e^{-z_i}}, \qquad \hat{y}_i = \mathbf{1}[p_i \geq \tau], \tag{2}$$

$$L_{\text{BCE}} = -\frac{1}{N}\sum_i \left[y_i \log p_i + (1-y_i)\log(1-p_i)\right], \tag{3}$$

$$L_{\text{Dice}} = 1 - \frac{2\sum_i p_i y_i + \epsilon}{\sum_i p_i + \sum_i y_i + \epsilon}, \qquad \epsilon = 10^{-6}, \tag{4}$$

$$L = L_{\text{BCE}} + \beta L_{\text{Dice}}. \tag{5}$$

BCE is computed through the stable logits formulation. Dice is computed separately for each image and averaged. The default decision threshold is $\tau = 0.5$; any change must use validation only. The mandatory baseline has $\beta = 0$.

Aggregate all pixels in the evaluated dataset into

$$C = \begin{pmatrix} TN & FP \\ FN & TP \end{pmatrix}, \qquad TP = \sum_i \mathbf{1}[\hat{y}_i = 1, y_i = 1], \tag{6}$$

with analogous definitions for true negatives, false positives and false negatives. Rows denote true class; columns denote predicted class. Then

$$\text{IoU}_{fg} = \frac{TP}{TP+FP+FN}, \qquad \text{IoU}_{bg} = \frac{TN}{TN+FP+FN}, \tag{7}$$

$$\text{mIoU} = \tfrac{1}{2}(\text{IoU}_{fg} + \text{IoU}_{bg}). \tag{8}$$

We assign an absent class (zero union) IoU of 1, representing correct absence. This is an explicit implementation convention because the PDF does not define that degeneracy. Per-image IoUs are never averaged to form the dataset score.

For foreground set M and four-connected neighborhood $\mathcal{N}_4$,

$$\partial M = \{i \in M : \exists j \in \mathcal{N}_4(i),\ j \notin M\}. \tag{9}$$

Pixels outside the image are background. Let $B_p = \partial \hat{M}$, $B_t = \partial M$, and $D_2(B)$ be pixels within a centered 5×5 window of B. Matching tolerance is Chebyshev distance two, including diagonal offsets $(2, 2)$. For one image,

$$P = \frac{|B_p \cap D_2(B_t)|}{|B_p|}, \qquad R = \frac{|B_t \cap D_2(B_p)|}{|B_t|}, \tag{10}$$

$$F_1 = \begin{cases} 2PR/(P+R) & P+R>0, \\ 0 & \text{otherwise}, \end{cases} \qquad BF1 = \frac{1}{K}\sum_{k=1}^{K} F_{1,k}. \tag{11}$$

Both empty boundaries yield $F_1 = 1$; exactly one empty boundary yields $F_1 = 0$. Four directional tensor slices extract the boundary. Neighborhood matching uses 25 whole-array boolean OR passes over zero-padded NumPy views, with no per-pixel Python loop or library morphology. Analytic corner/diagonal tests and equivalence checks against the permitted `max_pool2d` building block verify the exact window. Matching is many-to-many, as specified, rather than bipartite assignment. This optimization accelerates evaluation; metrics remain outside the inference timer.

For all quality metrics on the $[0, 1]$ scale,

$$Q = 0.70\,\text{mIoU} + 0.30 BF1, \tag{12}$$

$$CES = \begin{cases} Q\left[1 - 0.20\frac{T_{CPU}}{50} - 0.10\min\left(1, \frac{S_{disk}}{4194304}\right)\right], & T_{CPU} \leq 50, \\ 0, & T_{CPU} > 50. \end{cases} \tag{13}$$

T_{CPU} is in milliseconds and S_{disk} is the actual parameter-file size in bytes. For $Q = .7$, $T = 20$ and $S = 524288$, the exact result is .63525, consistent with the PDF's rounded .635.

6 Training methodology

The selected network was trained once for 16 epochs with AdamW (learning rate 3×10^{-3}, weight decay 10^{-4}), cosine learning-rate decay, batch size 16, gradient-norm clipping at 5 and pure BCE loss ($\beta = 0$), using the 9,498 training images at 128×128 internal resolution; the loss is computed on logits upsampled to 256×256. The best epoch by validation Q (epoch 15, $Q = 0.7877$ at $r = 128$) was kept. Training took about 10 minutes per epoch on the CPU listed below using 10 threads; the timed pipeline always uses one. Validation Q rose steadily from 0.51 (epoch 1) to 0.79 (epoch 16), see Figure 2. Each run stores its configuration, package versions, split hashes and per-epoch log.

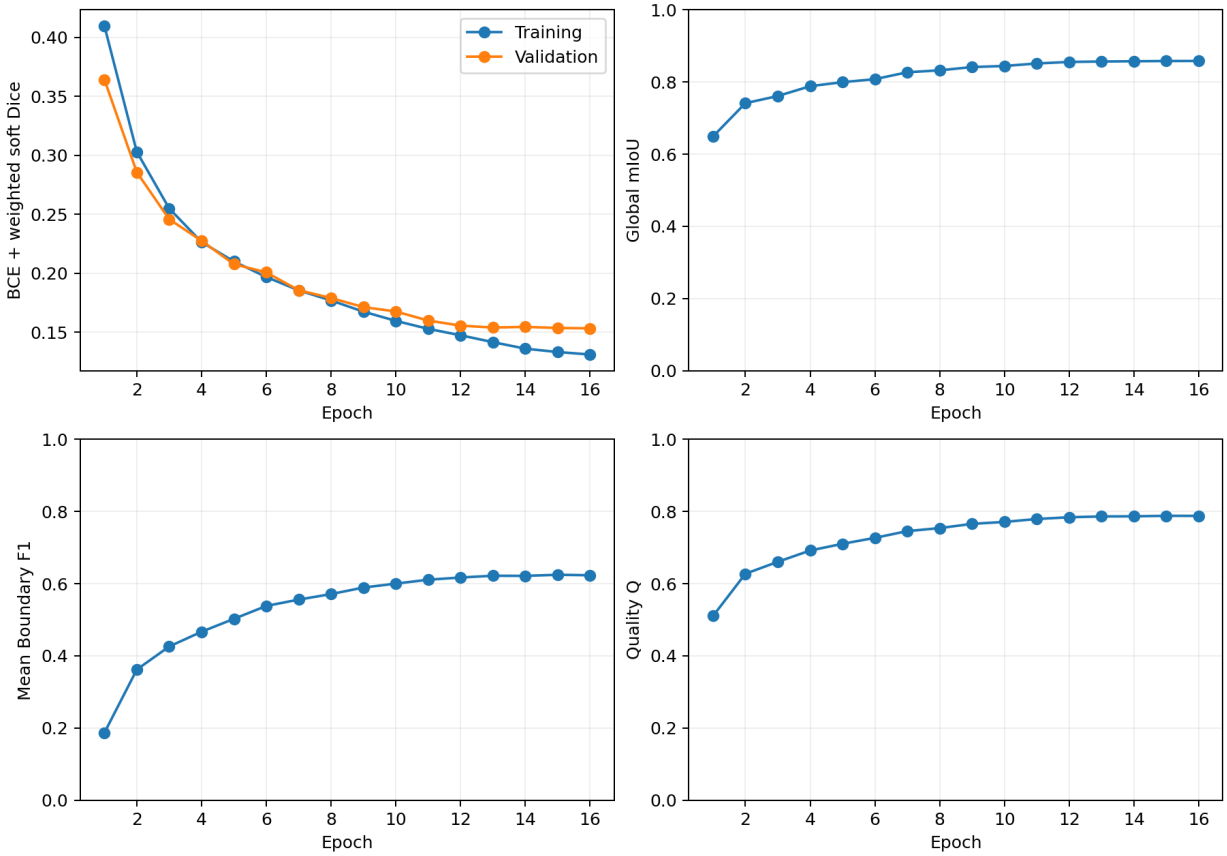


Figure 2: Training and validation curves of the selected run.

7 Computational efficiency and timing

The encoder operates at a configurable internal resolution; the full output remains 256×256. Standard blocks prioritize simple CPU kernels, while separable blocks reduce arithmetic at the possible cost of additional operator overhead. Single-thread execution is configured before NumPy, OpenCV or PyTorch imports through four required environment variables, followed by their thread-setting APIs. Inference uses CPU tensors, `eval()` and `inference_mode()`. No inference subprocess, data-loader worker or alternate engine is used.

One full warmup is excluded. For official timing, each DUTS-TE photo is loaded and resized outside the timer. The exact shared inference function is wrapped by `time.process_time()`. The sum of all 5,019 durations divided by 5,019 gives T_{CPU}. Metrics and file exports occur outside timing. The prediction CLI instead uses its output-producing pass as warmup, then

averages 20 additional full-pipeline runs. Size uses `os.path.getsize`; plain `torch.save` is not post-compressed.

8 Experimental comparison

Before the final run we ran three two-epoch pilots (Tiny FCN with BCE, Tiny FCN with BCE+Dice, depthwise-separable FCN with BCE; about 3–4k parameters each). They reached validation Q between 0.45 and 0.47, with BCE and BCE+Dice indistinguishable at this budget and the depthwise variant slower on CPU. We therefore moved to the larger U-Net and trained it with BCE only. No separate Dice run of the final model was made, so we make no claim about Dice at full training length.

Using the trained U-Net, we then varied the *inference* resolution on the validation set (Table 1). Because CES charges $0.2\,T/50$ for time, the smaller resolution wins even though Q is slightly lower: $r = 96$ gives CES 0.704 against 0.678 at $r = 128$. Resolutions above the training size lose quality. A threshold sweep from 0.4 to 0.6 at $r = 96$ changed Q by under 0.01, so the default 0.5 was kept. All rows come from `results/model_comparison.csv`. Individual validation timings there are noisy by a few ms because other programs ran concurrently (one 144-pixel run even shows 50.8 ms); the final timing in Table 2 is a clean 5,019-image measurement.

Inference r	mIoU	BF1	Q	Val. ms	CES
96	0.8503	0.5958	0.7739	11.9	0.7041
112	0.8580	0.6190	0.7863	18.8	0.6938
128	0.8576	0.6245	0.7877	24.1	0.6783
160	0.8368	0.5970	0.7648	42.6	0.6020

Table 1: Validation-only comparison of inference resolution for the trained U-Net.

9 Results

Set	mIoU	BF1	Q	CPU ms	Bytes	CES	N
Validation	0.8503	0.5958	0.7739	13.092	1783881	0.7005	1055
DUTS-TE	0.7049	0.3942	0.6117	16.873	1783881	0.5444	5019
Captured photos	Not evaluated; prerequisite evidence pending						

Table 2: Final results of the single frozen model. Time is measured per set with `time.process_time` on one thread; DUTS-TE is the official number. Captured-photo metrics are not yet available (see below).

The test CPU time is 16.9 ms, well inside the 50 ms limit, and the file is 1,783,881 bytes (0.43 % of 4 MiB), so the penalty terms cost only about 11% of Q. The gap between validation ($Q = 0.774$) and DUTS-TE ($Q = 0.612$) is large. Validation images come from the training distribution, whereas DUTS-TE contains many photographs with several objects, small objects and clutter, and the boundary score drops most (BF1 0.60 $\rightarrow$ 0.39): a 439k-parameter network working at 96×96 cannot localize fine contours. CPU: Intel(R) Core(TM) 7 150U. Actual saved split: 9498 training and 1055 validation images; seed 42.

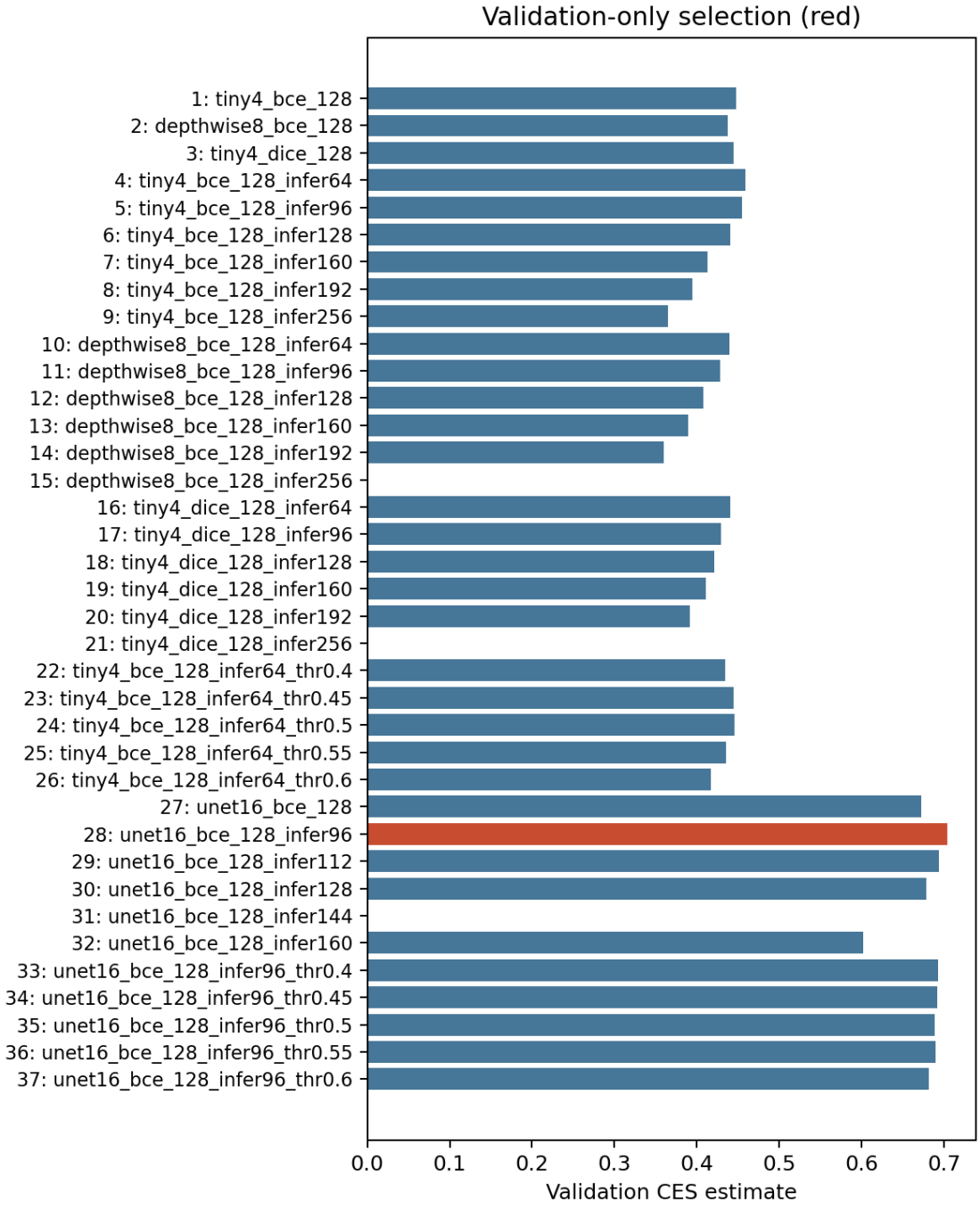


Figure 3: Validation CES of every measured configuration; the selected one is red.

10 Captured-photo results

Ten phone photographs (indoor and outdoor, near and far, cluttered scenes, a plant in a garden, bicycles in a shed) are stored in `captured/original` and resized to `captured/images_256`. **Their ground-truth masks have not been drawn yet**, so mIoU, BF1 and the true/predicted overlay pairs required by the assignment are not reported; the Captured row of Table 2 is therefore empty. Figure 4 shows predictions only (green overlay, photo on the left of each pair) as a qualitative preview.

11 Failure analysis

Two failure modes are visible in Figure 4. These are visual judgements, not measured IoU values, since ground truth is not available for these photos.

- **Clutter and many similar objects** (`cycle_inside`, `cycle_outside`, `study_table`). The photo contains a row of bicycles or a desk full of items and no single dominant object. The model marks only part of the nearest bicycle or an arbitrary desk region. DUTS teaches "salient" as a high-contrast, centred object; with many equal candidates the small network has too little global context to choose.
- **Small, low-contrast object** (`outside_plant`). The plants occupy a small area and resemble the soil in colour and texture, so almost nothing is segmented. At 96×96 internal resolution a small plant spans only a few pixels and gives weak features.

A milder issue is bleeding onto the supporting surface (`shoes`: the rug is included), probably because object and support share edges and colour statistics.

12 Conclusion and limitations

A 439k-parameter U-Net trained from scratch gives $Q = 0.612$ and CES $= 0.544$ on DUTS-TE at 16.9 ms and 1.78 MB, comfortably inside the compute budget. The score is limited by boundary accuracy and cluttered scenes. A larger model or higher input resolution could raise Q while staying under 50 ms; we did not explore this because one training run already took about three hours on CPU. Limitations: a single full training run, no Dice ablation at full length, no GPU, and captured-photo metrics still pending the hand-drawn masks.

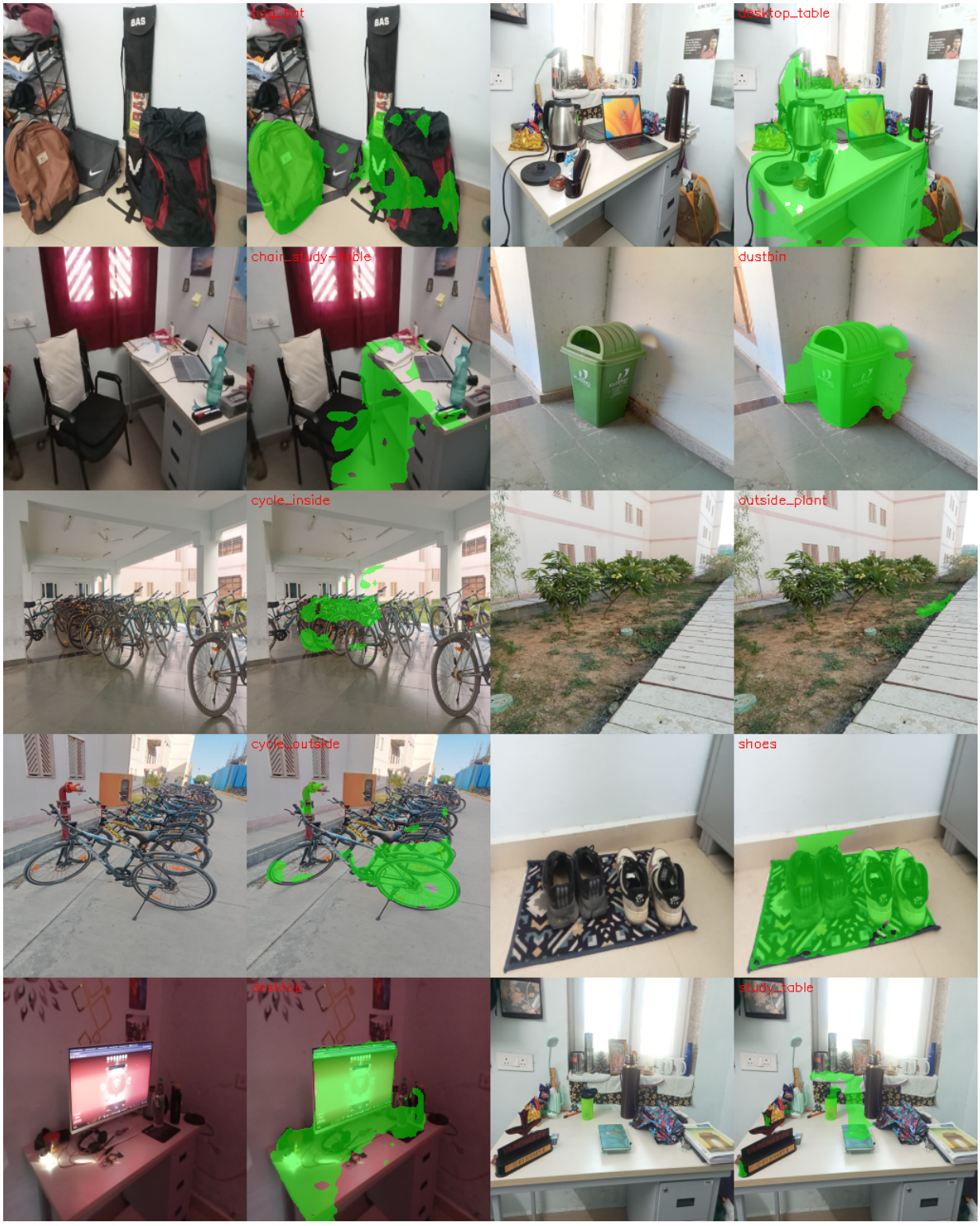


Figure 4: Predictions on the ten own photographs (no ground truth).

13 Reproducibility

Run these PowerShell commands in an outputs-empty copy containing source, configurations and genuine input data. Existing submitted weights/results should be preserved in a separate copy. Backticks continue a command on the next line. The complete selected configuration is `configs/final.json`; the training command below reproduces it without touching either test set (the training configuration is `configs/final_train.json`). Original comparison/sweep commands are documented in the root README and `docs/REPRODUCIBILITY.md`.

```
python -m pip install -r requirements.txt
python -m scripts.verify
python -m scripts.download_duts --split TR
python -m scripts.prepare_dataset --root data/DUTS
python train.py --config configs/final_train.json --out experiments/final/unet16_bce_128
# Reuses that completed run, evaluates validation and records selection:
python -m scripts.run_experiments --config configs/final.json `
  --experiments configs/reproduce_experiments.json --out experiments/reproduce
python -m scripts.freeze_model
$tr = 'data/DUTS/DUTS-TR'
python evaluate.py --split validation --images "$tr/DUTS-TR-Image" `
  --masks "$tr/DUTS-TR-Mask" --manifest data/splits/val.txt `
  --out results/validation_metrics.json
python -m scripts.download_duts --split TE
$te = 'data/DUTS/DUTS-TE'
# This is both official quality evaluation and CPU timing:
python benchmark.py --split duts-test --images "$te/DUTS-TE-Image" `
  --masks "$te/DUTS-TE-Mask" --out results/duts_test_metrics.json
python predict.py --image photo.jpg --out mask.png
# Only after authentic capture and manual annotation:
python -m scripts.verify_dataset --root captured --split captured
python evaluate.py --split captured --images captured/images_256 `
  --masks captured/masks_256 --out results/captured_metrics.json
python -m scripts.create_figures overlays `
  --images captured/images_256 --masks captured/masks_256 `
  --out results/captured_figures
python -m scripts.create_figures comparison
python -m scripts.build_report
python -m scripts.audit --submission
python -m scripts.package_submission
```

Prediction loads `weights/final_model.pth` and prints exactly `T_CPU_ms:` and `S_disk_bytes:`. The first pass writes the mask; the next 20 passes are timed. Report building supports pdflatex or an explicitly supplied portable Tectonic executable. Every compiled page requires visual review. Unavailable captured-photo evidence prevents the final audit from passing, even if all software tests pass.

References

[1] CSL7360, *Assignment 2: Cost-Efficient Segmentation under a Compute Budget*, supplied assignment PDF, pp. 1–3.

[2] L. Wang et al., *Learning to Detect Salient Objects with Image-level Supervision*, CVPR, 2017. Dataset: `https://saliencydetection.net/duts/`.